

K-means, Expectation-Maximisation and Boltzmann Machines

Learning probability distributions with hidden variables
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

24 September, 2026

What this lecture covers

Learning probabilities

Hidden variables and k-means

Expectation-maximisation (EM)

The Boltzmann machine

Learning as maximum likelihood

Summary

What this lecture covers

The colours carry meaning, and they are the same in every equation and every figure of these notes.

- ▶ **Blue** marks the parameters and the step that re-estimates them, the M-step: the means of k-means, and the observed nodes of the Boltzmann machine.
- ▶ **Orange** marks the hidden variables and the step that infers them, the E-step: the assignments of k-means, and the hidden nodes.
- ▶ **Red** marks the gap between a bound and what it bounds, which is a Kullback-Leibler divergence, and the free-running phase of Boltzmann learning.
- ▶ **Teal** marks the clamped phase of Boltzmann learning, where the data are held fixed, and the best of several options in a figure.

From Markov models to learning probability distributions

The last lecture built probability models by hand. A Markov random field puts a variable at every pixel, an energy scores every configuration with a single number that says how bad it is, and mean field theory estimates the hidden variables of that energy. Every constant in those models, the weight of the smoothness term and the price of a break, was chosen by the person who wrote the model down.

This lecture asks the other question: where do the numbers in a probability model come from? It is the question that Griffiths, Chater and Tenenbaum (2024) put at the centre of Bayesian accounts of cognition: a model is only as good as the parameters it is given, and those have to be learnt. Learning estimates the parameters of a distribution from training data. The first half of the lecture learns from data in which everything is observed; the rest learns when part of the data is missing, which is the situation in almost every vision problem.

The new question is: how do we learn these probability distributions, and can we learn them in ways that are neurally plausible?

From Markov models to learning probability distributions

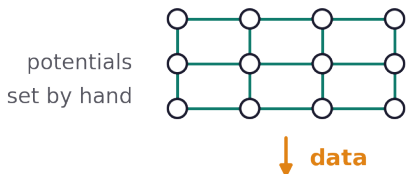
Three models are learnt in this lecture: k-means, which splits data into classes; the mixture of Gaussians, which is the probability model behind k-means; and the Boltzmann machine, a distribution over binary neurons. From a high level all three are variants of one algorithm, expectation-maximisation.

The weak membrane of the last lecture was already an instance of it. Its line processes were hidden variables, the distribution over those line processes that the last lecture wrote as B was the E-step of this lecture, and its free energy is the free energy written here as $\mathcal{F}(Q, \theta)$.

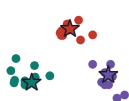
In the picture of the three models, each dot of the k-means panel is a datapoint and each star a mean; the circles of the mixture panel are one and two standard deviations of each Gaussian; and in the Boltzmann machine the blue nodes are observed, the white ones hidden, and the purple arcs are the links inside each layer.

From Markov models to learning probability distributions

last lecture: the models were given



this lecture: the models are learnt by EM



k-means



mixture of
Gaussians



Boltzmann
machine

Figure 1: The models of the last lecture, whose potentials were set by hand, and the three models of this one, whose parameters are learnt from data. In the first panel the teal grid is the Markov random field, one circle per pixel and one edge per neighbouring pair; the orange arrow marks the change of question, from potentials chosen by hand to parameters estimated from data.

Neuronally plausible learning is local, like Hebb's rule

A learning rule is called neuronally plausible when it can be carried out by local operations: the change of a synapse uses only quantities available at that synapse, which are the activities of the two neurons it joins. A rule that needed the activity of every neuron in the brain at once could not be implemented by a synapse.

Hebb's rule (1949) is the standard example. It says that a synapse strengthens when the neurons on its two sides are active together, which is often summarised as: neurons that fire together wire together. Write S_i for the activity of the neuron before the synapse and S_j for the activity of the neuron after it, and ω_{ij} for the strength of the synapse between them. Hebb's rule increases ω_{ij} whenever S_i and S_j are both active.

There is some empirical evidence for such a rule. Kandel's studies of the sea slug *Aplysia*, whose nervous system is small enough to record from cell by cell, showed that the strength of a synapse changes with the paired activity of the two cells (Kandel, 2001).

Neuronally plausible learning is local, like Hebb's rule

Most neural network learning algorithms can be formulated probabilistically, or closely approximated by a probabilistic formulation. That is the programme of this lecture: write down a probability model, learn its parameters by a principled criterion, and then ask whether the resulting update is local. The Boltzmann machine at the end of the lecture is the case where the answer is clearest: its learning rule contains Hebb's rule as its first term.

Neuronally plausible learning is local, like Hebb's rule

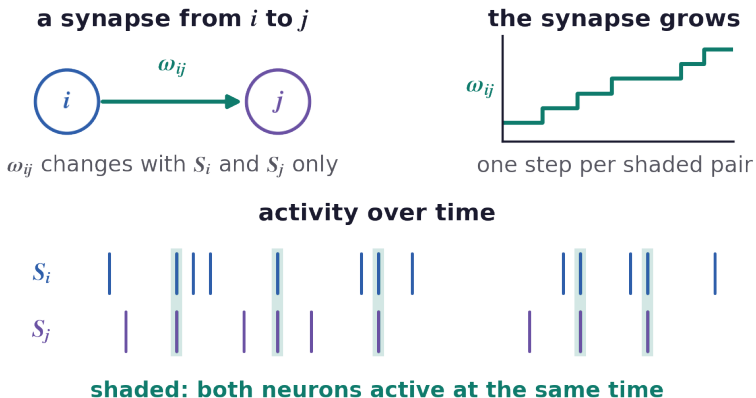


Figure 2: A schematic of Hebb's rule. The first panel is the synapse itself: the arrow runs from the neuron i before it to the neuron j after it, and its label is the strength w_{ij} . The spike trains are made up: each vertical bar is one spike of neuron i (blue) or neuron j (purple), time runs to the right, and a shaded column marks a moment when both neurons spike. The staircase is the weight w_{ij} , which rises by one step at every shaded column. Nothing outside the two trains enters the staircase, which is what local means.

Learning estimates the parameters α of a distribution

In Bayes decision theory (BDT) a problem is posed by writing down a probability distribution and then deciding what to do with it. Learning enters when the distribution is not known exactly: it is given a functional form with unknown constants, the parameters, and the parameters are estimated from data. Write $P(x | \alpha)$ for the model, where x is a datapoint and α stands for all the parameters.

The training data are n datapoints,

$$\mathcal{X} = \{x_1, \dots, x_n\},$$

assumed to be independent samples from the distribution.

Maximum likelihood makes the data most probable

The simplest criterion for choosing the parameters is maximum likelihood (ML): take the parameters under which the observed data have the highest probability,

$$\begin{aligned}\alpha^* &= \arg \max_{\alpha} P(\mathcal{X} | \alpha) = \arg \max_{\alpha} \prod_{i=1}^n P(x_i | \alpha) \\ &= \arg \min_{\alpha} \left\{ - \sum_{i=1}^n \log P(x_i | \alpha) \right\}.\end{aligned}$$

The product is over the datapoints because they are independent. The last form takes minus the logarithm, which turns the product into a sum and a maximisation into a minimisation; it is the form used in practice, because a product of n small numbers underflows and a sum of n logarithms does not.

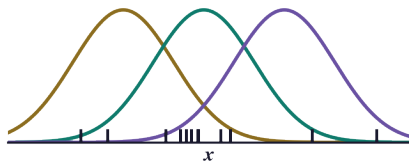
Maximum likelihood makes the data most probable

In the example of the figure the model is a Gaussian of known variance and the only parameter is its mean, $\alpha = \mu$. Twelve datapoints are drawn from it. Each candidate mean gives the data some probability, and the log-likelihood $\log P(\mathcal{X} | \mu)$ is a smooth curve with a single peak. The peak is at the sample mean $\mu^* = 1.10$, which is the maximum likelihood estimate for a Gaussian.

Learning of exactly this kind appeared in the edge detection lecture. The distributions of filter responses on edge and non-edge pixels were estimated from images in which a person had marked the edges, and those estimates are maximum likelihood estimates from annotated data.

Maximum likelihood makes the data most probable

three Gaussians for the same data



olive: μ too small

teal: $\mu = \mu^*$

purple: μ too large

the ticks on the axis are the 12 datapoints

the log-likelihood
peaks at μ^*

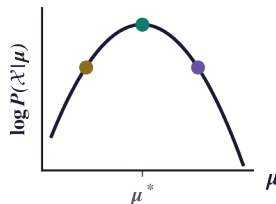


Figure 3: Maximum likelihood for the mean of a Gaussian. The ticks on the horizontal axis of the first plot are the twelve datapoints, and the three curves are Gaussians of the same variance with three different means. The second plot is the log-likelihood of the mean; the three dots are the three candidates, and the teal one sits at the peak.

Online learning updates α one datapoint at a time

For a Gaussian the maximum likelihood estimates have a closed form, the sample mean and the average squared deviation from it,

$$\mu^* = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma^{2,*} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu^*)^2.$$

This is batch learning: every datapoint is used at once, and the estimate is formed in one step from the whole data set.

Online learning updates α one datapoint at a time

Online learning does the same job one datapoint at a time. The standard method for it is stochastic gradient descent (SGD), which takes a step downhill on minus the log-likelihood of one datapoint, equivalently uphill on its log-likelihood. At time t a datapoint x_t is selected at random and the parameters take one small step uphill on its log-likelihood,

$$\alpha_{t+1} = \alpha_t + \zeta_t \frac{\partial \log P(x_t | \alpha_t)}{\partial \alpha},$$

where $\zeta_t > 0$ is the learning rate, a small number that may decrease slowly with t . The data are never held in memory all at once, which is why online learning is the more neurally plausible of the two, and why it is what large networks use.

Online learning updates α one datapoint at a time

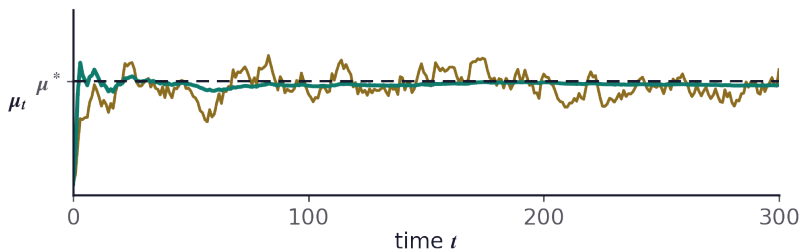
The learning rate has to be chosen with care. Robbins and Monro (1951) proved that the iteration converges when the rates are large enough to travel any distance and small enough to settle, which holds when $\sum_t \zeta_t$ diverges and $\sum_t \zeta_t^2$ converges. Their theorem also needs conditions on the model: that the noise has bounded variance, and that the derivative being chased crosses zero cleanly. Those conditions hold for the models here. The choice $\zeta_t = 1/(t+2)$ satisfies both; a constant rate satisfies the first and fails the second.

Online learning updates α one datapoint at a time

In the run plotted here the model is again a Gaussian mean, so that $\partial \log P(x | \mu) / \partial \mu = (x - \mu) / \sigma^2$ and, with $\sigma = 1$, the update is $\mu_{t+1} = \mu_t + \zeta_t (x_t - \mu_t)$: the estimate moves a fraction of the way towards the datapoint just seen. Starting from $\mu_0 = -1$ and drawing from a set of forty points whose batch estimate is $\mu^* = 0.88$, the decreasing rate settles near μ^* (it is at 0.81 after 300 draws) while the constant rate $\zeta = 0.15$ keeps bouncing around it (1.09 at $t = 300$).

The noise of the online estimate is not only a nuisance. On a criterion with several minima it lets the estimate escape a poor one, which is the reason the course notes say that online learning is more likely to find the correct parameters. That is a heuristic, not a theorem: nothing here guarantees the global optimum.

Online learning updates α one datapoint at a time



— ζ_t decreases with t : settles on the batch estimate

— constant ζ : keeps fluctuating

--- batch: μ^* from all the data

here $\alpha = \mu$ and $\sigma = I$: $\mu_{t+1} = \mu_t + \zeta_t(x_t - \mu_t)$

Figure 4: Online estimation of a Gaussian mean. The horizontal axis is the number of datapoints drawn so far and the vertical axis the current estimate μ_t . The teal trace uses $\zeta_t = 1/(t+2)$ and the olive trace a constant $\zeta = 0.15$; the dashed line is the batch estimate μ^* from all forty points.

The prior matters most when there is little data

Maximum likelihood uses the data alone. If something is known about the parameters before any data arrive, that knowledge is a prior $P(\alpha)$ and the criterion becomes the posterior,

$$P(\alpha | \mathcal{X}) \propto P(\mathcal{X} | \alpha) P(\alpha),$$

whose maximiser is the maximum a posteriori (MAP) estimate. Taking minus the logarithm again, the quantity to minimise is

$$-\sum_{i=1}^n \log P(x_i | \alpha) - \log P(\alpha) :$$

n terms from the likelihood and one term from the prior.

The prior matters most when there is little data

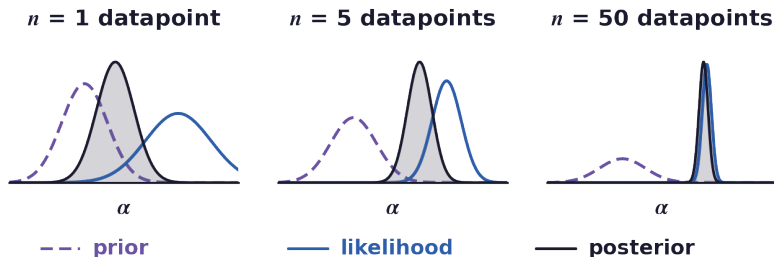
The counting is the heart of it. The likelihood contributes n terms and the prior one, so with many datapoints the prior is outvoted and the estimate is essentially the maximum likelihood estimate; with few datapoints the single prior term is a large fraction of the total and pulls the estimate towards what was believed beforehand. A good prior is therefore worth most when data are scarce: it lets you learn from less data.

The prior matters most when there is little data

In the three panels the parameter is again a mean. The prior is a Gaussian centred at -0.5 , the data come from a Gaussian centred at 2 , and the posterior is drawn for $n = 1, 5$ and 50 datapoints. Its peak is at $0.44, 1.52$ and 1.98 : with one datapoint the prior wins, with fifty it hardly matters.

Two further remarks from the 2025 lecture notes for this course (Yuille, 2025), called the course notes from here on. A loss function is sometimes necessary as well, particularly for learning conditional distributions, because the cost of different mistakes need not be the same. And choosing the functional form of $P(x|\alpha)$, rather than its parameters, is beyond the scope of this course; model selection, which searches over a space of models for the one that best explains the data, is one strategy for it.

The prior matters most when there is little data



the posterior moves from the prior towards the likelihood as n grows

Figure 5: The prior, the likelihood and the posterior of a mean, for three sizes of data set. In each panel the horizontal axis is the parameter α ; the dashed purple curve is the prior, which is the same in all three, the blue curve is the likelihood, which sharpens as n grows, and the shaded black curve is the posterior. Each panel is scaled to its own tallest curve, so the prior looks shorter as the posterior sharpens.

Maximum likelihood minimises the KL divergence to the data

Maximum likelihood has a second reading, which relates it to the cross-entropy loss used to train classifiers. Treat the training data themselves as a probability distribution: put weight $1/n$ on each datapoint and nothing anywhere else,

$$P_{\mathcal{X}}(x) = \frac{1}{n} \sum_{i=1}^n \delta(x - x_i),$$

where δ is the Kronecker delta, so $P_{\mathcal{X}}$ puts a spike of height $1/n$ at each observed value, and twice that where a value occurs twice. In the example plotted here the data are grey levels, which take sixteen discrete values, so the sums over x below are finite sums.

Maximum likelihood minimises the KL divergence to the data

The Kullback-Leibler (KL) divergence measures how far the model is from this data distribution,

$$KL(\alpha) = \sum_x P_{\mathcal{X}}(x) \log \frac{P_{\mathcal{X}}(x)}{P(x|\alpha)}.$$

It is never negative, and it is zero only when the two distributions agree.

Maximum likelihood minimises the KL divergence to the data

Splitting the logarithm of the ratio gives two sums,

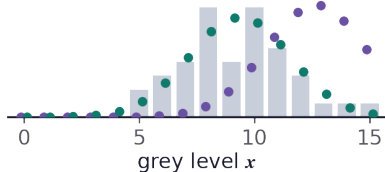
$$KL(\alpha) = \underbrace{\sum_x P_{\mathcal{X}}(x) \log P_{\mathcal{X}}(x)}_{\text{independent of } \alpha} \quad - \underbrace{\frac{1}{n} \sum_{i=1}^n \log P(x_i | \alpha)}_{\text{minus the average log-likelihood}}$$

where the second sum turned into an average over the datapoints because $P_{\mathcal{X}}$ puts weight $1/n$ on each of them and zero elsewhere. The first term depends on the data alone, so it is a constant as far as the parameters are concerned.

Minimising the KL divergence over α is therefore the same as maximising the likelihood. The second term is the cross-entropy of the model under the data distribution, which is the loss that classifiers are trained with. In the figure the two curves differ by the constant $\sum_x P_{\mathcal{X}} \log P_{\mathcal{X}} = -2.18$ and reach their minimum at the same $\mu^* = 9.24$.

Maximum likelihood minimises the KL divergence to the data

the data $P_{\mathcal{X}}$ and two models



bars: $P_{\mathcal{X}}$, the data

dots: the model $P(x|\mu)$

red: the KL divergence

blue: minus the average log-likelihood

teal dots: $\mu = \mu^*$

purple dots: μ too large

the same minimum

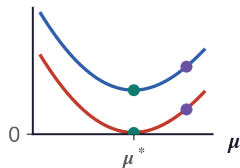


Figure 6: Forty grey levels and a Gaussian model of them restricted to the sixteen levels. In the first plot the bars are the data distribution $P_{\mathcal{X}}$ and the dots the model for two values of its mean. The second plot is the KL divergence and minus the average log-likelihood as functions of that mean: two curves a constant apart, with the same minimiser μ^* . The KL divergence bottoms out just above zero, not at it, because a Gaussian cannot reproduce a histogram of forty samples exactly.

Deep networks learn conditional distributions $P(y | x, \alpha)$

Everything so far learns a distribution over one variable. The same machinery learns a conditional distribution from pairs. The training data are now n pairs (x_i, y_i) , an input and its answer, and the criterion is

$$\alpha^* = \arg \max_{\alpha} \prod_{i=1}^n P(y_i | x_i, \alpha) = \arg \min_{\alpha} \left\{ - \sum_{i=1}^n \log P(y_i | x_i, \alpha) \right\}.$$

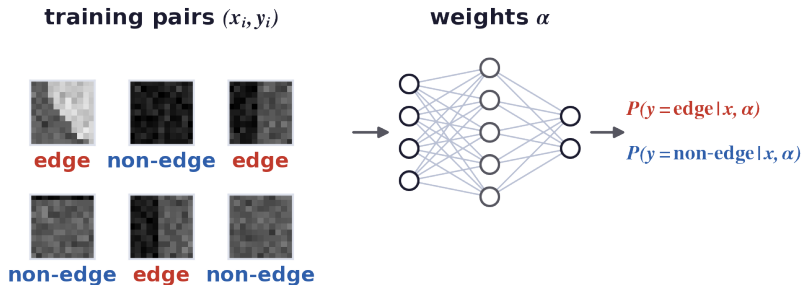
No distribution over the inputs x is needed, because every term is conditioned on the input that came with it.

For a neural network the parameters α are the weights of the network, and the functional form of $P(y | x, \alpha)$ is whatever the architecture computes: AlexNet, ResNet and the transformers are different choices of that form. Both numbers are large, and in modern networks the weights often outnumber the training pairs, so the sum is never formed in one batch. What is used instead is mini-batch learning, a compromise between the two extremes: each step uses a small subset of the data rather than one datapoint or all of them.

Deep networks learn conditional distributions $P(y \mid x, \alpha)$

Edge detection is the example carried over from the earlier lecture. The input x is a window of the image around a pixel and the answer y says whether that pixel lies on an edge. Networks trained for this task need loss functions that penalise a missed edge more than a false one, because edge pixels are a small fraction of all pixels and a network trained on plain likelihood can do well by answering non-edge everywhere.

Deep networks learn conditional distributions $P(y | x, \alpha)$



each patch is an 11×11 window; its label is the label of the centre pixel
the network turns a patch x into a distribution over the two labels y

Figure 7: Training pairs for edge detection and the network that learns from them. Each patch is an 11×11 window of a synthetic noisy image, labelled edge when its centre pixel lies on a region boundary. A patch labelled edge is marked in red and one labelled non-edge in blue. The circles in the middle are units of the network and the grey lines its weights α ; the number of units drawn is schematic. The two output units, one per label, carry the two numbers the network produces for one patch, in the colour of the label each stands for.

Hidden variables are missing from the training data

Learning is harder when the training data do not specify everything the model talks about. A variable of the model that is never observed is called hidden, or latent, or missing; it has to be inferred at the same time as the parameters are estimated.

The situation was met independently in three fields, which is why it has three names. Statisticians noticed that real tables have gaps: a survey has unanswered questions, a medical record has tests that were not run, and the analysis has to work with what is there. Engineers building speech recognisers used hidden Markov models, in which a sequence of states generates the sounds and only the sounds are recorded; hidden there means not given by the training data. In vision the line processes of the weak membrane were hidden variables: the model needs them to describe where the surface breaks, and no training set ever marks them.

Hidden variables are missing from the training data

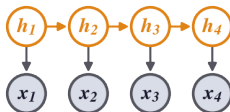
Statisticians formulated the algorithm for this situation and called it expectation-maximisation (Dempster, Laird and Rubin, 1977). Artificial intelligence researchers later re-formulated the same algorithm as the minimisation of a free energy (Neal and Hinton, 1998), which is the form used in this lecture and the form in which the weak membrane used it.

Hidden variables are missing from the training data

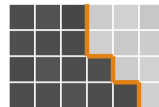
**statistics:
missing data**



**speech: hidden
Markov models**



**vision:
line processes**



?: a missing entry orange: the hidden states orange: the breaks
in each case the training data leave some variables unspecified

Figure 8: The same idea in three fields. In the statistics panel each cell of the table is one recorded value and a question mark is a gap. In the speech panel the orange circles are the hidden states of a hidden Markov model, the horizontal arrows between them the transitions, and the grey circles the observed sounds, each reached by a downward arrow from the state that emitted it. In the vision panel each square is a pixel, dark and light marking two regions of different brightness, and an orange segment on the side shared by two pixels marks a break between them.

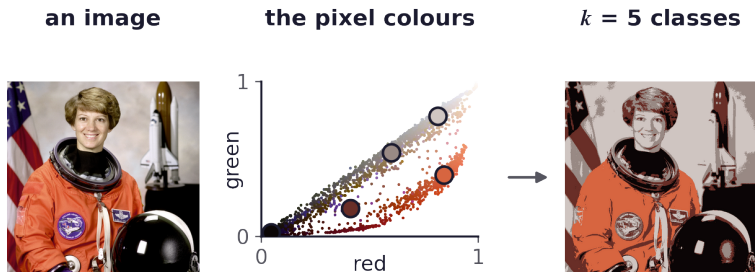
K-means clusters data into classes without labels

K-means is the simplest example of learning with hidden variables, and it is a hard-assignment version of the expectation-maximisation (EM) algorithm. It takes a set of datapoints and splits them into k classes. Nothing tells it which class any datapoint belongs to: the class is the hidden variable. Learning without labels of this kind is called unsupervised learning.

The example of the figure is colour quantisation. Each pixel of a photograph is a point in three dimensions, its red, green and blue values, and the hidden variable is which of five colour classes that pixel belongs to. K-means finds five representative colours and every pixel is then painted with the representative of its class.

K-means is not only a classical clustering method. Neural networks, and transformers in particular, can contain it as a subcomponent: kMaX-DeepLab (Yu et al., 2022) rewrites one step of a transformer, the step that matches each image pixel to one of a set of slots standing for the objects of the scene, as a k-means assignment of the pixels to those slots.

K-means clusters data into classes without labels



each dot is one pixel in its own colour; the circles are the five means
the third panel repaints every pixel with the mean of its class

Figure 9: K-means with $k = 5$ on the colours of the pixels of a photograph. The middle panel plots a sample of the pixels in the red-green plane, each dot drawn in its own colour, and the five circles are the means found; blue is not plotted, so a mean can look displaced from the cloud it belongs to. The third panel is the photograph with every pixel replaced by the mean of its class (photograph: NASA, public domain).

K-means minimises an energy of assignments and means

The input is a set of unlabelled datapoints $D = \{x_i\}$: the same training set the first section called $\mathcal{X}$, renamed here because the subset belonging to each class needs a letter of its own. It is decomposed into k disjoint classes, with k known in advance. Two kinds of unknown appear. Each class a has a mean m_a , a point of the same space as the data, and each datapoint has an assignment: the hidden variable V_{ia} is 1 when datapoint x_i belongs to class a and 0 otherwise. Every datapoint belongs to exactly one class, so

$$V_{ia} \in \{0, 1\}, \quad \sum_{a=1}^k V_{ia} = 1 \quad \text{for every } i :$$

one 1 in each row of the table of assignments.

K-means minimises an energy of assignments and means

The criterion is an energy, a single number that scores a whole solution,

$$E(\{V\}, \{m\}) = \sum_{i=1}^n \sum_{a=1}^k V_{ia} |x_i - m_a|^2 = \sum_{a=1}^k \sum_{x \in D_a} |x - m_a|^2,$$

where $D_a = \{x_i : V_{ia} = 1\}$ is the set of datapoints of class a . Because each row of V has a single 1, every datapoint contributes the squared distance to the mean of its own class and nothing else, which is what the second form makes explicit.

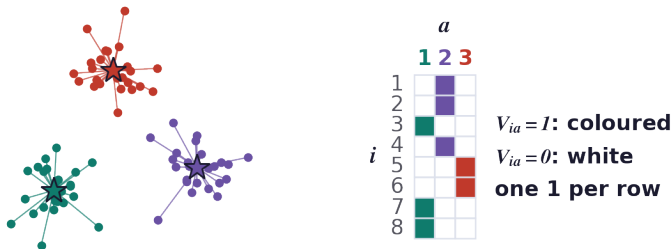
K-means minimises an energy of assignments and means

A low energy means tight classes: the datapoints of a class lie close to their mean. Minimising E over the assignments and the means together is the whole problem.

It is not an easy problem. E is not convex in its unknowns, and the assignments are discrete, so there is no gradient to follow in them. No efficient algorithm is known that finds the global minimum in general. The k-means algorithm settles for a local minimum: a solution that cannot be improved by either of its two moves, although a different starting point may reach a lower one.

K-means minimises an energy of assignments and means

each x_i joined to its mean



E adds the squared lengths of the lines: each x_i pays $(x_i - m_a)^2$
the table shows V for eight of the 72 datapoints, one row each

Figure 10: Seventy-two datapoints in two dimensions, in three classes. The star of each colour is that class's mean m_a , and every datapoint is joined to the mean of its class by a line of that class's colour, so the energy is the sum of the squared lengths of all the lines; here $E = 38.8$, a number the picture does not show. The table is the assignment matrix V for eight of the datapoints: one row per datapoint, one column per class, and the single coloured cell of a row is the class that datapoint belongs to.

The best mean of a fixed class is its average

Before the algorithm, one calculation. Minimise E over the means with the assignments held fixed. The answer is worth deriving once, because the same calculation returns as step 2 of k-means, in soft k-means, and in the M-step of EM.

With V fixed, the energy splits into one independent term per class, and the term of class a is $\sum_{x \in D_a} |x - m_a|^2$. Differentiating it with respect to m_a gives

$$\frac{\partial}{\partial m_a} \sum_{x \in D_a} |x - m_a|^2 = -2 \sum_{x \in D_a} (x - m_a) = 0 \implies m_a = \frac{1}{|D_a|} \sum_{x \in D_a} x,$$

where $|D_a|$ is the number of datapoints in class a . The mean of a class is the average of its members, and since the second derivative in m_a is $2|D_a|$ times the identity, which is positive definite, this stationary point is the minimum.

The best mean of a fixed class is its average

The same quantity can be written with the assignment matrix in place of the set, $m_a = (\sum_i V_{ia})^{-1} \sum_i V_{ia} x_i$, because $\sum_i V_{ia}$ counts the datapoints of class a and V_{ia} selects them. The two forms are the same number.

The k-means algorithm alternates two steps

The algorithm has four steps and repeats two of them.

Step 1 initialises the algorithm. The simplest choice assigns every datapoint to a class at random, which is a partition. A better one is k-means++ (Arthur and Vassilvitskii, 2007), which picks the first mean at random and then picks each further mean from the data with probability proportional to its squared distance to the nearest mean already chosen, so the starting means are spread out. That choice supplies means rather than a partition, so a run seeded with it begins at step 3.

The k-means algorithm alternates two steps

Step 2 computes the mean of each class,

$$m_a = \frac{1}{|D_a|} \sum_{x \in D_a} x,$$

and step 3 reassigns every datapoint to the class whose mean is nearest,

$$d_a(x_i) = |x_i - m_a|^2, \quad a^* = \arg \min \{d_1(x_i), \dots, d_k(x_i)\},$$

which sets $V_{ia^*} = 1$ and the rest of that row to zero. Step 4 repeats steps 2 and 3 until the assignment stops changing.

The k-means algorithm alternates two steps

Two details decide what the algorithm does in the awkward cases. If a datapoint is equally near two means the assignment rule is ambiguous, so a fixed tie rule, for example the smaller index, keeps exactly one 1 in each row. If a class loses all its datapoints its mean is undefined and has to be re-initialised, for instance at the datapoint furthest from its own mean.

Step 3 gives the classes a geometric shape. Assigning each point to the nearest mean divides the space into regions, one per mean, with straight boundaries half-way between neighbouring means. The run in the figure starts from a random partition, whose three means all sit near the centre of the data, and reaches its final assignment after four iterations; its energy falls 286.0, 240.2, 152.6, 134.3, 111.8, 72.6, 41.7, 39.6, 38.8 along the way, one value per half-step.

The k-means algorithm alternates two steps

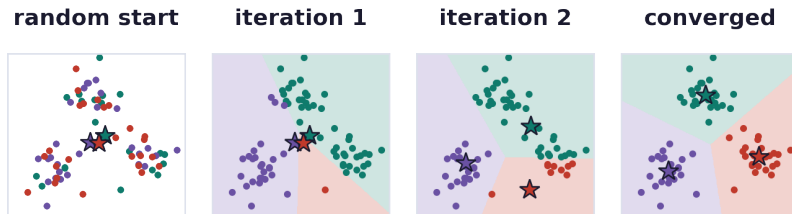


Figure 11: Four stages of one run. In each panel the stars are the means computed by step 2, the colours of the dots are the assignment made by step 3 with those means, and the shaded regions are the parts of the plane closest to each star. The first panel is the random partition and the means it gives, before any assignment has been made, which is why it carries no shading; those three means lie almost on top of one another.

Each step lowers the energy, down to a local minimum

Both repeated steps are exact minimisations of the same energy over one of its two arguments:

$$m_a = \frac{1}{\sum_i V_{ia}} \sum_i V_{ia} x_i \quad \text{minimises } E \text{ over } \{m\} \text{ with } \{V\} \text{ fixed,}$$

$$V_{ia} = 1 \text{ if } |x_i - m_a|^2 = \min_b |x_i - m_b|^2, \quad V_{ia} = 0 \text{ otherwise}$$

minimises E over $\{V\}$ with $\{m\}$ fixed, ties broken by the smaller index so that each row still holds a single 1, because with the means held fixed each datapoint's term is smallest when it is assigned to its nearest mean, and the terms of different datapoints do not interact.

Each step lowers the energy, down to a local minimum

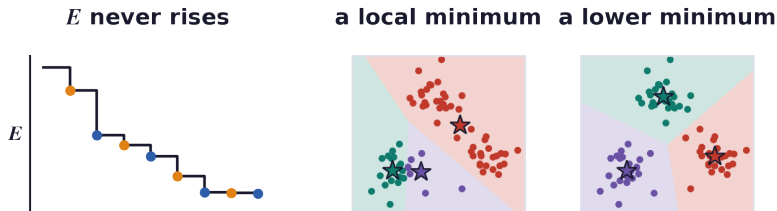
Neither step can raise E , so the sequence of energies is non-increasing. There are finitely many assignments, and the means are fixed by the assignment, so an assignment that has been visited once cannot be improved on when it is met again: the algorithm stops the first time the assignment repeats. It stops at a local minimum: a configuration that neither step can improve. There is no guarantee that this is the lowest energy available.

Each step lowers the energy, down to a local minimum

Two runs on the same seventy-two datapoints, from different starting partitions, show what the lack of a guarantee costs. One ends with two means inside a single cluster and the third stranded between the two remaining clusters, at $E = 145.6$; the other ends with one mean per cluster, at $E = 38.8$. Both are local minima and neither run can escape its own.

The usual remedy is to run k-means several times from different starting partitions and keep the solution with the lowest energy, which is why the initialisation of step 1 is worth some care.

Each step lowers the energy, down to a local minimum



orange dot: after step 3, the assignment

blue dot: after step 2, the means

the two runs start from different partitions of the same data

Figure 12: The staircase is the energy of one run after every half-step, with a blue dot after each step 2 and an orange dot after each step 3; it never rises. The run starts from the means of a random partition, so the first half-step recorded is a step 3 and the first dot is orange. The two scatter plots are the final states of two runs from different starting partitions, each region shaded by the star it is closest to.

Online k-means is a winner-take-all neural network

K-means can be written as a small neural network, which is where the claim of neuronal plausibility is cashed in; the network is the competitive-learning circuit of Hertz, Krogh and Palmer (1991). The network has k output units, one per class. Unit k has a vector of weights $m_{k,a}$, one weight for each component a of the input; these weights are its receptive field, the part of the input it is sensitive to together with the pattern there that drives it hardest, and they play the part of the mean of class k . Two letters are doing double duty here. The letter a indexes the components of the input vector, not the class as it did in the k-means energy; and k is both the number of classes and the index of one output unit, so a sum or a condition written for all k runs over all the units.

Online k-means is a winner-take-all neural network

An input vector x arrives on the input units. Each output unit forms the weighted sum of the input,

$$\sum_a m_{k,a} x_a,$$

and the units inhibit one another so strongly that only the one with the largest sum stays active. This is a winner-take-all circuit: the winning unit k^* satisfies

$$\sum_a m_{k^*,a} x_a \geq \sum_a m_{k,a} x_a \quad \text{for all } k, \quad |x| = |m_k| = 1.$$

The condition $|x| = 1$ is a condition on the data, not only on the weights: the circuit is given inputs of unit length. That is not a restriction in practice, because image patches are routinely normalised to remove the effect of overall brightness and contrast, which is exactly the assumption the von Mises-Fisher model makes about data on the unit sphere.

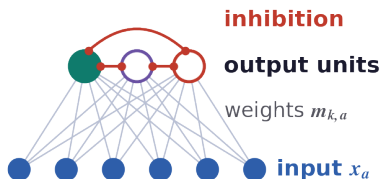
Online k-means is a winner-take-all neural network

The condition on the lengths matters. Expanding the squared distance gives $|x - m_k|^2 = |x|^2 - 2x \cdot m_k + |m_k|^2$, so the mean nearest to x is the one with the largest dot product only when all the $|m_k|$ are equal; a unit with a long weight vector would otherwise win on length alone. With all the vectors of unit length the largest dot product and the nearest mean are the same unit, and the circuit performs step 3 of k-means.

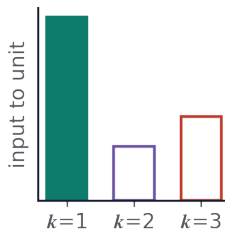
An alternative is to leave the lengths free and give each unit a bias of $-|m_k|^2/2$, so that the unit with the largest $x \cdot m_k - |m_k|^2/2$ wins; that quantity is minus half the squared distance plus a term independent of k .

Online k-means is a winner-take-all neural network

the network



the biggest input fires



the input of unit k is $\sum_a m_{k,a} x_a$; only the largest one is left active

Figure 13: The network. The blue circles at the bottom are the input units, one per component of x ; the grey lines are the weights $m_{k,a}$; the three circles above are the output units, joined by red inhibitory connections. The bar chart is the weighted sum arriving at each of the three units for one input: the first unit wins and is drawn filled.

The winning neuron moves its weights towards the input

The network so far only assigns. Learning is the other half: the weights have to become the means. Online k-means does this by steepest descent on the k-means energy, one input at a time, which is stochastic gradient descent on E .

When input x arrives and unit k^* wins, the only term of the energy that involves this input is $\sum_a (x_a - m_{k^*,a})^2$. Its derivative with respect to $m_{k^*,a}$ is $-2(x_a - m_{k^*,a})$, so a step of size ζ downhill is

$$m_{k^*,a} \mapsto m_{k^*,a} - \zeta 2 (m_{k^*,a} - x_a),$$

and the weights of every other unit are unchanged. The winner moves a fraction of the way towards the input, and the losers stay where they are; with $\zeta = 0.25$ the winner moves half-way.

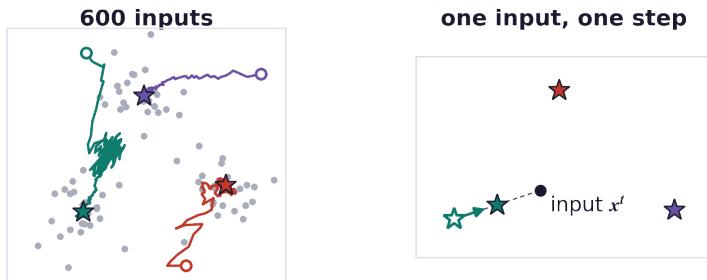
The winning neuron moves its weights towards the input

This is local once the winner is known: the change of a weight uses the input arriving on that synapse and the weight's own current value, and nothing else. It is not Hebb's rule, which uses the product of the two activities; it is that rule with a decay term. What is not local is the choice of the winner, and that is what the inhibitory circuit is for: the comparison is made by the neurons themselves rather than read out from outside.

The winning neuron moves its weights towards the input

One point needs care. Renormalising the winner after each step, which the dot-product competition needs in order to keep the weights of unit length, is not a step of steepest descent on E over unconstrained means: its fixed point is the normalised sum of the inputs of a class rather than their plain average. That variant is spherical k-means, and it is the algorithm behind the von Mises-Fisher model of data on the unit sphere. So the circuit as described, with unit-length inputs and renormalised weights, runs spherical k-means throughout; plain k-means is the version with the lengths left free and the bias $-|m_k|^2/2$ supplying what the normalisation would have.

The winning neuron moves its weights towards the input



open circle: the start; line: the path; star: the end

the nearest mean moves towards the input

the other two means stay where they are

Figure 14: Online k-means. In the first panel the grey dots are the seventy-two datapoints, the open circles are the three starting weight vectors, the lines are the paths they follow over 600 randomly drawn inputs with $\zeta_t = 0.08/(1 + t/150)$, and the stars are where they end. The second panel is a single update with $\zeta = 0.25$: the open star is the winner before the step, the black dot the input, and the filled star the winner afterwards. The two other filled stars are the losing units, which do not move.

Soft k-means assigns datapoints to clusters with probabilities

K-means makes a hard decision: each datapoint belongs to exactly one class. A datapoint half-way between two means is forced to choose, and that choice is what makes the energy a function of discrete assignments. Soft k-means replaces the binary assignment by a probability. Softening removes the discreteness; it does not remove the local optima, and soft k-means still stops at a different answer from a different start.

Soft k-means assigns datapoints to clusters with probabilities

Write ω_a for the event that a datapoint belongs to class a , following The course notes and the slides. This is the second job the letter ω does in this lecture: with two subscripts, ω_{ij} , it is the synaptic weight of Hebb's rule and of the Boltzmann machine; with one, ω_a , it names a class. The two never appear on the same frame. The soft assignment of datapoint x_j is a distribution over the classes,

$$P(\omega_a | x_j) = \frac{\exp\{-\frac{1}{2\sigma^2}|x_j - m_a|^2\}}{\sum_b \exp\{-\frac{1}{2\sigma^2}|x_j - m_b|^2\}},$$

a number between 0 and 1 for each class, summing to one over the classes because of the denominator. The index b runs over all the classes.

Soft k-means assigns datapoints to clusters with probabilities

The means are then the weighted averages

$$m_a = \frac{\sum_j x_j P(\omega_a | x_j)}{\sum_j P(\omega_a | x_j)},$$

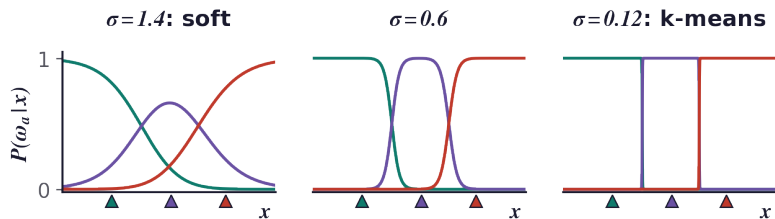
which is the calculation of step 2 with the hard count $|D_a|$ replaced by the soft count $\sum_j P(\omega_a | x_j)$.

Soft k-means assigns datapoints to clusters with probabilities

The constant σ^2 controls how soft the assignment is. When σ^2 is large the exponentials are all close to one and every datapoint is shared out almost equally; when σ^2 is small the nearest mean dominates the sum, and in the limit the probability is 1 for the nearest mean and 0 for the others, provided the nearest mean is unique. Soft k-means with $\sigma^2 \rightarrow 0$ is k-means, with the same tie rule needed as before.

The algorithm is otherwise unchanged: initialise, then alternate the soft assignment and the weighted means until nothing moves. Starting from a partition rather than from means, the soft version needs the means of that partition before its first assignment step.

Soft k-means assigns datapoints to clusters with probabilities



triangles: the three means m_a ; each curve is one cluster
the three curves sum to one at every x

Figure 15: The soft assignment of three classes on a line, for three values of σ . The horizontal axis is the datapoint x and the vertical axis the probability $P(\omega_a | x)$; each curve is one class, and the triangles mark the three means. The three values of σ are comparable to the gaps between the means, then smaller, then much smaller; as σ falls the curves approach steps. Taking σ far larger than the gaps, which is not drawn, would flatten all three curves towards $1/3$.

Soft k-means assumes a mixture of Gaussians

Soft k-means looks like a smoothed heuristic, and it is in fact the exact algorithm for a probability model. The model says that the data were generated by a mixture of Gaussians: to make a datapoint, first choose one of k classes, then draw a point from the Gaussian of that class, which has mean m_a and covariance $\sigma^2 I$ with I the identity matrix.

The variable that records the choice is the assignment V of the k-means energy. Given the assignments, the data are independent Gaussian draws,

$$P(\mathcal{X} | \{V\}, \{m\}) = \frac{1}{Z} \exp \left\{ - \sum_{ia} V_{ia} \frac{|x_i - m_a|^2}{2\sigma^2} \right\},$$

$$P(x_i | V, m) = \mathcal{N} \left(x_i : \sum_a V_{ia} m_a, \sigma^2 I \right),$$

where Z is the normalising constant of the Gaussians, and $\mathcal{N}(x : m, \Sigma)$ denotes the Gaussian density in x with mean m and covariance Σ . The sum $\sum_a V_{ia} m_a$ picks out the mean of the class that generated x_i , because exactly one V_{ia} is 1.

Soft k-means assumes a mixture of Gaussians

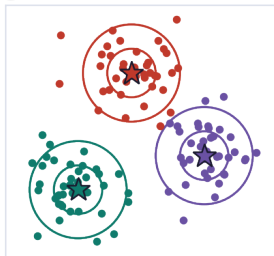
The model is not complete until the assignments themselves have a distribution. The natural choice, when nothing distinguishes the classes in advance, is the uniform prior: every one of the k^n possible assignments of n datapoints to k classes is equally probable, which is the same as saying that each datapoint chooses its class uniformly and independently.

The mixture is what is left when the assignments are summed out. Only then is the model a distribution over the data alone, and the sum is what makes maximum likelihood awkward, which is the difficulty EM was built to remove.

This is the equal-weight, shared-covariance special case of the mixture of Gaussians. The general model gives each class its own mixing weight π_a and its own covariance matrix and estimates those as well. Both steps then change: the normalising constants no longer cancel out of the responsibilities, so they stay in the E-step, and the M-step gains an update for the weights and the covariances.

Soft k-means assumes a mixture of Gaussians

generated: V known



hide V



observed: V hidden



choose a component, then draw a point from its Gaussian ($m_a, \sigma^2 I$)
circles at one and two σ ; the training data are the grey points alone

Figure 16: The mixture model. In the first panel the colour of each dot is the class that generated it, the star is that class's mean and the circles are at one and two standard deviations of each Gaussian. In the second panel the same points are drawn in grey: that is what the training data look like, and the orange arrow between the panels marks what the learner is not given, the assignment V .

EM estimates parameters when some variables are hidden

The general problem is stated with three symbols. The observed data are x , the hidden variables are h , and the parameters to be learnt are θ . The model gives the joint distribution $P(x, h | \theta)$ of the observed and the hidden variables together.

Two letters have changed. In the first section x was one datapoint and the whole training set was $\mathcal{X}$; here x stands for all the observed data at once, and h for all the hidden variables at once, because the sums below run over both together. And the parameters are written θ rather than α , following the usual statement of EM, which needs a letter of its own for the hidden variables.

EM estimates parameters when some variables are hidden

Maximum likelihood needs a distribution over the observed data alone, so the hidden variables are summed out,

$$P(x | \theta) = \sum_h P(x, h | \theta) = \sum_h P(x | h, \theta) P(h | \theta),$$

$$\theta^* = \arg \max_{\theta} \log \sum_h P(x, h | \theta).$$

The sum is over every value the hidden variables can take. The course notes write $P(h)$ for a prior that does not depend on θ , which is right for the mixture; these notes write $P(h | \theta)$, which also covers the Boltzmann machine, whose hidden nodes have a distribution that depends on the weights, and the weak membrane, whose prior couples the line processes to the surface.

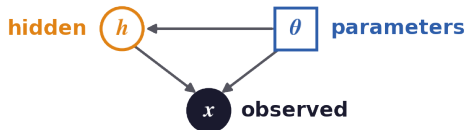
EM estimates parameters when some variables are hidden

The difficulty is the logarithm of a sum. When nothing is hidden the log-likelihood is a sum of n separate terms, one per datapoint, and each term can be differentiated on its own. Here the logarithm sits outside a sum over the hidden values, so the derivative in θ of one hidden value depends on every other hidden value through the denominator, and the terms no longer separate. The logarithm of the joint $P(x, h | \theta)$ does separate, and that is the quantity the M-step will work with.

EM was designed for exactly this (Dempster, Laird and Rubin, 1977). From a high level, k-means and learning mixtures or Boltzmann machines are all variants of it, which is why one algorithm accounts for the whole lecture.

EM estimates parameters when some variables are hidden

the general model: two causes of the data



EM in three models of the course

	hidden h	parameters θ
mixture of Gaussians	which component	the means $\{m\}$
weak membrane	the line processes	the surface
Boltzmann machine	the hidden nodes	the weights

an arrow points from a cause to what it explains: h and θ together shape x

Figure 17: The general model and the three instances of it used in this course. In the graph an arrow points from a cause to what it explains: h and θ together shape the observed x , and θ also shapes the distribution of h . The table names h and θ in each model. The weak membrane of the last lecture is in the list, in place of k-means, because its line processes are hidden variables; there the smoothed surface plays the part of θ , being the quantity minimised over while the line processes are averaged out, even though it is estimated afresh for every image rather than learnt once.

The EM algorithm alternates an E-step and an M-step

EM starts from a guess θ^0 and repeats two steps. The names come from the two operations: an expectation and a maximisation.

The E-step replaces the missing variables by probabilities. With θ held fixed at its current value, it computes the posterior of the hidden variables given the data,

$$Q^{t+1}(h) = P(h | x, \theta^t),$$

a distribution over the values of h , written Q because it will later be allowed to be any distribution at all.

The EM algorithm alternates an E-step and an M-step

The M-step then holds Q fixed and chooses the parameters that maximise the expected log-likelihood of the complete data (x, h) , the expectation being taken under Q ,

$$\theta^{t+1} = \arg \max_{\theta} \sum_h Q^{t+1}(h) \log P(x, h | \theta).$$

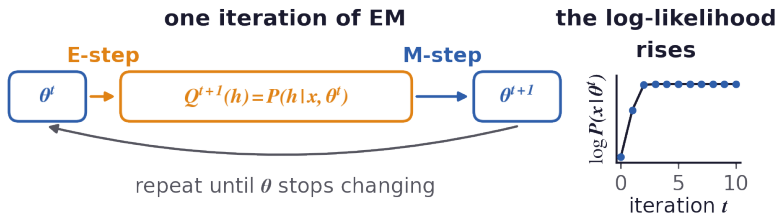
The logarithm is now inside the sum, so the terms separate and the maximisation is usually easy; for the mixture it is the weighted means of soft k-means.

The EM algorithm alternates an E-step and an M-step

When the data are n independent datapoints the pairs (x_j, h_j) are independent of each other, so the posterior of the hidden variables given the data and the parameters factorises, and with it Q , into one distribution $Q_j(h_j)$ per datapoint and the E-step is done separately for each.

The guarantee that makes EM useful is that the log-likelihood $\log P(x | \theta^t)$ never decreases from one iteration to the next. Being non-decreasing and bounded above, the sequence of log-likelihoods converges. What converges is that sequence of numbers: the parameters themselves need not settle, and the point they approach is a stationary point of the likelihood, in practice a local maximum but in principle a saddle. Different starting values reach different ones, and nothing here finds the global maximum.

The EM algorithm alternates an E-step and an M-step



the E-step fills in the hidden variables; the M-step re-estimates θ
the plot is the mixture example: 120 points, three means, $\sigma=0.5$
the log-likelihood never falls, whatever the starting θ^0

Figure 18: One iteration of EM, and the log-likelihood of the mixture example over ten iterations. The example is the one used for every EM figure of these notes: 120 datapoints drawn from three Gaussians with $\sigma = 0.5$, with the means started near the centre of the data. The log-likelihood rises -670.0 , -435.4 , -308.2 , -304.0 , -303.98 and then stays put.

Every EM step moves uphill or stays level

The guarantee follows from one identity, which is also the identity behind the free energy of EM. For any distribution Q on the hidden variables,

$$\log P(x|\theta) = \sum_h Q(h) \log \frac{P(x, h|\theta)}{Q(h)} + \underbrace{\sum_h Q(h) \log \frac{Q(h)}{P(h|x, \theta)}}_{KL(Q, P(h|x, \theta)) \geq 0},$$

which is checked by writing $P(x, h|\theta) = P(h|x, \theta)P(x|\theta)$ in the first sum and using $\sum_h Q(h) = 1$. Call the first sum $L(Q, \theta)$. Since the KL divergence is never negative, $L(Q, \theta)$ is a lower bound on the log-likelihood for every Q , and the two are equal exactly when $Q(h) = P(h|x, \theta)$.

The bound is tightened, then raised

The E-step sets $Q^{t+1}(h) = P(h|x, \theta^t)$, which makes the bound tight at the current parameters: $L(Q^{t+1}, \theta^t) = \log P(x|\theta^t)$. The M-step maximises $\sum_h Q^{t+1}(h) \log P(x, h|\theta)$ over θ , which is the same as maximising $L(Q^{t+1}, \theta)$ because the $-\sum Q \log Q$ part of L does not involve θ . Putting the three facts in a row,

$$\log P(x|\theta^{t+1}) \geq L(Q^{t+1}, \theta^{t+1}) \geq L(Q^{t+1}, \theta^t) = \log P(x|\theta^t).$$

The first step is the bound, the second is the M-step, the third is the E-step.

EM for a mixture of Gaussians is soft k-means

Apply the two steps to the mixture model. The parameters are the means, $\theta = \{m_a\}$; the covariance $\sigma^2 I$ is fixed and the k classes have equal weight $1/k$. The hidden variable of datapoint x_j is which class generated it.

The E-step is Bayes rule for that one datapoint,

$$P(\omega_a | x_j) = \frac{P(x_j | \omega_a)}{\sum_b P(x_j | \omega_b)} \quad \text{with} \quad P(x_j | \omega_a) = \mathcal{N}(x_j : m_a, \sigma^2 I).$$

The equal weights $1/k$ appear in the numerator and in every term of the denominator and cancel, as do the normalising constants of the Gaussians, which are the same for every class because σ is shared. What is left is exactly the soft assignment of soft k-means.

EM for a mixture of Gaussians is soft k-means

The M-step maximises $\sum_j \sum_a P(\omega_a | x_j) \log P(x_j, \omega_a | m)$ over the means. Only one term involves m_a ,

$$\sum_j P(\omega_a | x_j) \log \mathcal{N}(x_j : m_a, \sigma^2 I) = \text{const} - \sum_j P(\omega_a | x_j) \frac{|x_j - m_a|^2}{2\sigma^2},$$

and setting its derivative in m_a to zero gives $\sum_j P(\omega_a | x_j)(x_j - m_a) = 0$, hence

$$m_a = \frac{\sum_j x_j P(\omega_a | x_j)}{\sum_j P(\omega_a | x_j)},$$

the weighted mean of soft k-means. The two algorithms are the same algorithm.

As σ^2 goes to zero the posterior of the E-step puts all its mass on the nearest mean, whenever that mean is unique, and the weighted mean becomes a plain average over the class. EM with hard assignments is k-means.

EM for a mixture of Gaussians is soft k-means



Figure 19: Four stages of EM on 120 datapoints from three Gaussians with $\sigma = 0.5$, the true value being given to the algorithm. The stars are the current means and the colour of each dot is its soft assignment to them, blended from the three class colours, so a grey dot is one that is shared. The three means end at $(-0.12, 0.05)$, $(2.63, 0.79)$ and $(1.12, 2.45)$ against the true $(0, 0)$, $(2.6, 0.7)$ and $(1.1, 2.4)$.

Normalised data lie on the unit sphere: von Mises-Fisher

A second mixture is worth writing down, because it is the model behind the winner-take-all network and because it is the natural model for image patches.

The brightness of an image patch depends on the lighting as much as on what is in the patch: a patch I seen under different illumination becomes $cI + d$, with a contrast c and a background d . Subtracting the mean of the patch removes d , and dividing by the length of the result removes c . What is left is a vector of unit length: only the direction of the patch vector survives, and the direction is what carries the pattern.

On the unit sphere the Gaussian exponent turns into a dot product. For unit vectors $|x - m_k|^2 = 2 - 2x \cdot m_k$, so

$$-\frac{|x - m_k|^2}{2\sigma_k^2} = \frac{x \cdot m_k - 1}{\sigma_k^2},$$

and the $-1/\sigma_k^2$ is a constant that the normalisation absorbs.

Normalised data lie on the unit sphere: von Mises-Fisher

Writing $\lambda_k = 1/\sigma_k^2$ gives the von Mises-Fisher distribution,

$$P(x | k, \lambda_k) = \frac{\exp\{\lambda_k m_k \cdot x\}}{Z(\lambda_k)}, \quad |x| = |m_k| = 1,$$

where k labels one component of the mixture, m_k is its mean direction, λ_k is its concentration, large for a tight cluster, and $Z(\lambda_k)$ normalises the density over the sphere. The course notes call the constant σ_k ; the constant of the standard von Mises-Fisher distribution is λ_k .

EM fits a mixture of these in the same way as a mixture of Gaussians (Banerjee et al., 2005). With a single concentration λ shared by all the components, the limit $\lambda \rightarrow \infty$ assigns each datapoint to the component with the largest dot product, which is spherical k-means, and is what the winner-take-all network with normalised weights computes. With different λ_k the rule keeps each component's own constants and reads $\arg \max_k \{\lambda_k m_k \cdot x - \log Z(\lambda_k)\}$.

Normalised data lie on the unit sphere: von Mises-Fisher

The picture has three parts. The first is one 8×8 patch at three contrasts together with the single unit vector they all reduce to, which is what normalising throws away. The second is a two-dimensional sketch of three von Mises-Fisher components on the circle: the dashed line of each is its mean direction m_k and the bump around it is its density, wide for a small λ_k and narrow for a large one.

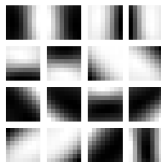
The third is the sixteen means of spherical k-means run on 20,000 patches of a photograph, each mean drawn as an 8×8 image. They are oriented edges and bars: the receptive fields that the winner-take-all network would learn.

Normalised data lie on the unit sphere: von Mises-Fisher

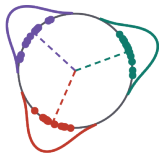
one patch, three contrasts



16 means: edges and bars



unit-length data



dashed: the mean direction m_k of each cluster
curves: the three densities on the circle
dots: patches drawn from them
spherical k-means on 20,000 patches

the contrast c changes the length of the patch vector, not its direction
so the direction carries the pattern: the means are oriented edges and bars

Figure 20: Normalised patches, the von Mises-Fisher density on the circle, and the means found by spherical k-means.

EM minimises a free energy of θ and Q alternately

The identity used to prove the guarantee can be turned around and used as the definition of the algorithm. Let $Q(h)$ be any distribution over the hidden variables, not necessarily the posterior, and define the free energy

$$\begin{aligned}\mathcal{F}(Q, \theta) &= \underbrace{-\log P(x | \theta)}_{\text{minus the log-likelihood}} + \underbrace{KL(Q, P(h | x, \theta))}_{\text{a gap, never negative}} \\ &= \underbrace{-\sum_h Q(h) \log P(x, h | \theta)}_{\text{expected energy}} + \underbrace{\sum_h Q(h) \log Q(h)}_{\text{minus the entropy}}.\end{aligned}$$

The two forms are equal for every Q and every θ . The first is the identity behind the monotonicity proof with the sign changed, so

$\mathcal{F}(Q, \theta) = -L(Q, \theta)$, and the second follows from it by writing $P(x, h | \theta) = P(h | x, \theta)P(x | \theta)$.

EM minimises a free energy of θ and Q alternately

The second form is the free energy of the last lecture: an expected energy minus an entropy, with the energy of a configuration being $-\log P(x, h | \theta)$. The entropy of Q is $-\sum_h Q(h) \log Q(h)$, a measure of how spread out Q is, at its largest when Q is uniform and zero when Q puts all its weight on one value. The free energy of the last lecture multiplied the entropy by a temperature; here the two terms carry equal weight.

EM minimises a free energy of θ and Q alternately

Both steps of EM are minimisations of this one function.

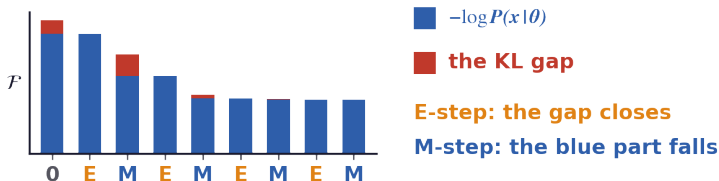
The E-step minimises $\mathcal{F}$ over Q with θ fixed. In the first form only the KL term involves Q , and a KL divergence is minimised, at zero, when its two arguments agree. So the minimising Q is the posterior $P(h|x, \theta)$, and at that Q the free energy equals minus the log-likelihood.

The M-step minimises $\mathcal{F}$ over θ with Q fixed. In the second form only the expected energy involves θ , so minimising it is maximising $\sum_h Q(h) \log P(x, h|\theta)$, which is the M-step.

Neither half-step can raise $\mathcal{F}$, so $\mathcal{F}$ falls monotonically and EM stops at a local minimum of it. Since $\mathcal{F} \geq -\log P(x|\theta)$ always, with equality after every E-step, the log-likelihood cannot decrease either.

EM minimises a free energy of θ and Q alternately

the free energy after each half-step



0 is the start; the bar is $\mathcal{F}(Q, \theta)$, the blue part minus the log-likelihood
an E-step closes the red gap; an M-step lowers the blue part
no bar is ever taller than the one before it

Figure 21: The free energy of the mixture example after every half-step, split into its two parts. The height of a bar is $\mathcal{F}(Q, \theta)$; the blue part is $-\log P(x|\theta)$ and the red part the KL gap. At the start $\mathcal{F} = 747.6$, of which 670.0 is minus the log-likelihood and the rest the KL gap; the first E-step closes that gap, leaving 670.0; the first M-step drops the blue part to 435.4 and opens a gap of 119.7; and so on down to 304.0.

Partial E- and M-steps also lower the free energy

Nothing in the argument required either step to be an exact minimisation. All that was used is that neither step raises $\mathcal{F}$:

$$\mathcal{F}(Q^{t+1}, \theta^{t+1}) \leq \mathcal{F}(Q^{t+1}, \theta^t) \leq \mathcal{F}(Q^t, \theta^t).$$

A step that merely lowers $\mathcal{F}$ over θ , for example a few gradient steps instead of a full maximisation, keeps the inequality. EM with such a step is called generalised EM, and the Boltzmann machine is learnt this way.

The two relaxations are not equally harmless. With exact E-steps the bound is tight at the start of each M-step, so the log-likelihood still cannot decrease however partial the M-step is. With an approximate E-step, such as the mean field approximation of the last lecture, which restricts Q to a family that cannot contain the posterior, the gap never closes; $\mathcal{F}$ still falls, but it is only an upper bound on $-\log P(x | \theta)$, and the log-likelihood itself can go down. On the mixture example, moving Q twenty per cent of the way from uniform towards the posterior and then taking an exact M-step lowers $\mathcal{F}$ from 747.6 to 724.4 to 714.4 while the log-likelihood falls from -670.0 to -683.3 .

Partial E- and M-steps also lower the free energy

EM stops at a local optimum in any of these versions, and which one it reaches depends on where it starts, exactly as for k-means. Two starts on the same data end at log-likelihoods -396.2 and -189.2 .

The scatter plots use a second, smaller data set: seventy-two points from the same three Gaussians with $\sigma = 0.5$, so its free energies are not comparable with the numbers quoted for the 120-point example. Its first plot runs an exact M-step against a partial one that moves each mean thirty per cent of the way to its weighted mean; both fall at every half-step and both reach the same optimum. That is the harmless relaxation. The dangerous one is the partial E-step that moves Q only part of the way to the posterior, which is not drawn.

Partial E- and M-steps also lower the free energy

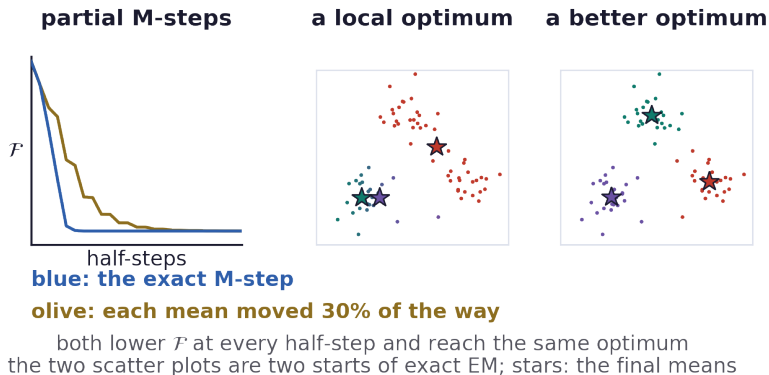


Figure 22: The free energy after every half-step for two runs from one start, the exact M-step in blue and the partial one in olive; both reach $\mathcal{F} = 189.2$, the partial one more slowly. The two scatter plots are the ends of two runs of exact EM from different starts, with the stars the final means and the colours the soft assignments.

The Boltzmann machine is a Gibbs distribution on neurons

The last model of the lecture is a distribution over the states of a network of neurons. Each neuron i has a binary activity $S_i \in \{0, 1\}$, off or on, and the state of the whole machine is the vector $\vec{S}$ of all of them. Every pair of neurons is joined by a symmetric weight $\omega_{ij} = \omega_{ji}$, with no weight from a neuron to itself; these weights are the parameters to be learnt.

The Boltzmann machine is a Gibbs distribution on neurons

The state is scored by an energy, and the energy defines a Gibbs distribution exactly as in the last lecture. A Gibbs distribution is one of the form $e^{-E/T}/Z$: the lower the energy of a state, the more probable it is. The constant Z , here and everywhere in this lecture, is whatever makes the distribution in front of it sum to one; for a Gibbs distribution it also has a name of its own, the partition function.

The Boltzmann machine is a Gibbs distribution on neurons

$$\underbrace{E(\vec{S}) = -\frac{1}{2} \sum_{ij} \omega_{ij} S_i S_j}_{\text{the energy}}, \quad \underbrace{P(\vec{S}) = \frac{1}{Z} \exp\{-E(\vec{S})/T\}}_{\text{the Gibbs distribution}},$$

$$\underbrace{Z = \sum_{\vec{S}} \exp\{-E(\vec{S})/T\}}_{\text{the normalisation}}.$$

The double sum runs over every ordered pair, so each pair of neurons appears twice and the factor $\frac{1}{2}$ counts it once. A positive ω_{ij} makes it cheap for the two neurons to be on together, and a low energy means a high probability.

The Boltzmann machine is a Gibbs distribution on neurons

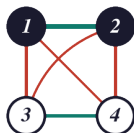
The temperature T sets how sharply the distribution prefers low energy. When T is small the exponential magnifies energy differences and almost all the probability sits on the lowest-energy state; when T is large the differences are flattened and the states approach equal probability. It plays the part a variance plays in a Gaussian.

The normalisation Z is a sum over every state of the machine, of which there are $2^{|\mathcal{V}|}$ for a machine of $|\mathcal{V}|$ neurons. The letter N is kept for the number of training examples, which the learning section needs. For the four neurons of the figure that is sixteen terms and can be written out; for a machine of any useful size it cannot be computed, and every algorithm in this lecture works around that fact.

Bias terms, which would let a neuron prefer to be on or off on its own, are left out here as in the course notes. A bias is nothing new: it is a weight to an extra neuron whose activity is always 1.

The Boltzmann machine is a Gibbs distribution on neurons

four neurons



teal: $\omega_{ij} > 0$

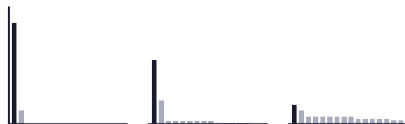
red: $\omega_{ij} < 0$

black: $S_i = 1$

$T = 0.5$

$T = 1$

$T = 3$



bars: $P(\vec{S})$ over the 16 states, lowest energy first
the black bar is the lowest-energy state, so it comes first
a small T concentrates the probability on that state
a large T spreads it over all 16

Figure 23: A machine of four neurons with $\omega_{12} = 3$, $\omega_{34} = 2$, $\omega_{13} = \omega_{24} = -2$ and $\omega_{14} = \omega_{23} = -1$, and its Gibbs distribution at three temperatures. The three bar charts have the sixteen states along the horizontal axis, ordered by energy, the lowest first, the lowest drawn black and the rest grey. In the graph a teal edge is a positive weight and a red one a negative weight. The lowest state is 1100, with neurons 1 and 2 on, drawn filled in the graph; its probability is 0.87, 0.55 and 0.17 at $T = 0.5$, 1 and 3.

Gibbs sampling updates one neuron at a time

Two things are wanted from the distribution. Inference asks for the most probable state $\vec{S}^*$, and learning will ask for averages under $P(\vec{S})$. Both are sums or maxima over the $2^{|\mathcal{V}|}$ states of the machine and neither can be computed directly.

The most probable state is the state of lowest energy,

$$\vec{S}^* = \arg \max_{\vec{S}} P(\vec{S}) = \arg \min_{\vec{S}} E(\vec{S}),$$

because the exponential is decreasing and Z is the same for every state. Two approximations were given in the last lecture: Gibbs sampling with the temperature lowered slowly, which is simulated annealing, and mean field theory.

Gibbs sampling updates one neuron at a time

Gibbs sampling initialises the state at random, selects one neuron i at random, and replaces its value by a draw from its conditional distribution given all the others, written $\vec{S}/i$,

$$P(S_i | \vec{S}/i) = \frac{\exp\{S_i \sum_{j \neq i} \omega_{ij} S_j / T\}}{1 + \exp\{\sum_{j \neq i} \omega_{ij} S_j / T\}},$$

and repeats. Only the chosen neuron changes at each step, and it may well be redrawn to the value it already had.

The conditional of one neuron is a sigmoid

The conditional used by the sampler is worth deriving, because it is the only place where the intractable Z disappears for free. Fix every neuron but i and compare the two states that differ in S_i . Every term of the energy that does not contain S_i is the same in both, and the terms that do contain it are $-S_i \sum_{j \neq i} \omega_{ij} S_j$, where the factor $\frac{1}{2}$ has gone because the pair ij appears twice in the double sum.

The conditional of one neuron is a sigmoid

Writing $u_i = \sum_{j \neq i} \omega_{ij} S_j$ for the total input to neuron i , the two unnormalised probabilities are $\exp\{u_i/T\}$ for $S_i = 1$ and $\exp\{0\} = 1$ for $S_i = 0$. Dividing by their sum,

$$P(S_i=1 \mid \vec{S}/i) = \frac{e^{u_i/T}}{1 + e^{u_i/T}} = \frac{1}{1 + e^{-u_i/T}},$$

which is the sigmoid of the input divided by the temperature. The constant Z cancels because it appears in both probabilities, so one neuron can be resampled at the cost of a sum over its neighbours.

A low temperature makes the sigmoid steep, so the neuron almost always takes the value its input prefers; a high temperature flattens it towards one half.

Gibbs sampling updates one neuron at a time

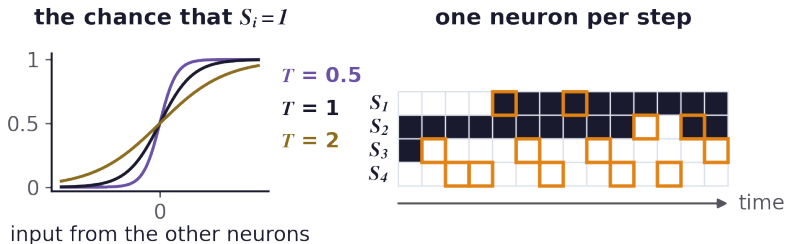


Figure 24: The conditional and thirteen steps of a chain on the four-neuron machine. The first plot is the sigmoid at three temperatures, with the total input on the horizontal axis. In the second the four rows are the four neurons and a black square means $S_i = 1$. The first column is the state the chain was started at and carries no outline; each of the thirteen columns after it is one step, and its orange outline marks the neuron redrawn at that step.

Averages over Gibbs samples approximate expectations

At a fixed temperature the chain converges to samples from $P(\vec{S})$: a state that is twice as probable is visited about twice as often. It converges because at any finite temperature every state has positive probability, so the chain can reach all of them from wherever it starts. Its first steps still remember the random state it was started at, and that opening stretch is discarded; it is called the burn-in. The chain is not a search for the most probable state, and for a machine with many neurons the samples need not lie anywhere near it, because a large number of moderately probable states can carry far more probability than one peak. Successive samples are also correlated, since only one neuron changes per step.

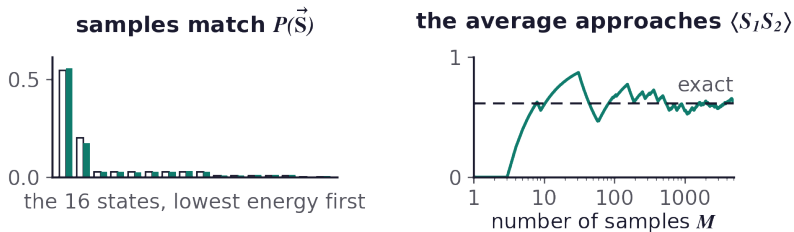
Averages over Gibbs samples approximate expectations

What the samples are good for is expectations. For any function of the state, the average over the samples approaches the expectation under P . The case needed for learning is the correlation of two neurons,

$$\langle S_i S_j \rangle = \sum_{\vec{S}} S_i S_j P(\vec{S}) \approx \frac{1}{M} \sum_{s=1}^M S_i^s S_j^s,$$

where $\vec{S}^1, \dots, \vec{S}^M$ are M samples of the chain and S_i^s is the value of neuron i in sample s . Since both neurons are binary, $S_i S_j$ is 1 exactly when both are on, so this correlation is the fraction of the time the two neurons are on together. The word correlation is used here as physics uses it, for an average of a product: nothing is subtracted off and nothing is divided by a standard deviation.

Averages over Gibbs samples approximate expectations



white bars: $P(\vec{S})$; teal bars: the fraction of 5000 Gibbs samples
the right plot is the running average of $S_1 S_2$ over the samples
the dashed line is the exact expectation

Figure 25: Five thousand Gibbs samples of the four-neuron machine at $T = 1$, after two hundred steps of burn-in. In the first plot the white bars are the exact $P(\vec{S})$ and the teal bars the fraction of the samples in each state; half the total absolute difference between the two, summed over the sixteen states, is 0.03. The second plot is the running average of $S_1 S_2$ against the number of samples used, on a logarithmic scale; it ends at 0.634 against the exact 0.614.

Learning fits the observed nodes to the training data

Learning a Boltzmann machine means choosing the weights. The nodes are divided into two groups: the observed nodes $\vec{S}_o$, which are where the data are presented, and the hidden nodes $\vec{S}_h$, which have no data of their own and exist to let the machine represent structure that the observed nodes alone could not. The state of the whole machine is $\vec{S} = (\vec{S}_o, \vec{S}_h)$ and its distribution is the Gibbs distribution $P(\vec{S}) = \exp\{-E(\vec{S})/T\}/Z$.

Learning fits the observed nodes to the training data

What the machine says about the data is the marginal of the observed nodes, the distribution left after the hidden nodes are summed out,

$$P(\vec{S}_o) = \sum_{\vec{S}_h} \frac{1}{Z} \exp\{-E(\vec{S})/T\}.$$

The training data give a distribution $R(\vec{S}_o)$ over the observed states, and learning makes $P(\vec{S}_o)$ as close as possible to $R(\vec{S}_o)$ in the KL divergence,

$$KL(\vec{\omega}) = \sum_{\vec{S}_o} R(\vec{S}_o) \log \frac{R(\vec{S}_o)}{P(\vec{S}_o)},$$

which is a function of the weights through P alone and is never negative.

Learning fits the observed nodes to the training data

Boltzmann machines have been used as models of cortex as well as learning algorithms. Zhang, Li, Samonds and Lee (2016) fitted one to the disparity signals of natural three-dimensional scenes and compared the weights it learnt with the functional connectivity measured between disparity-tuned neurons in primary visual cortex, which connects this model back to the cooperative stereo model of the last lecture.

Learning fits the observed nodes to the training data

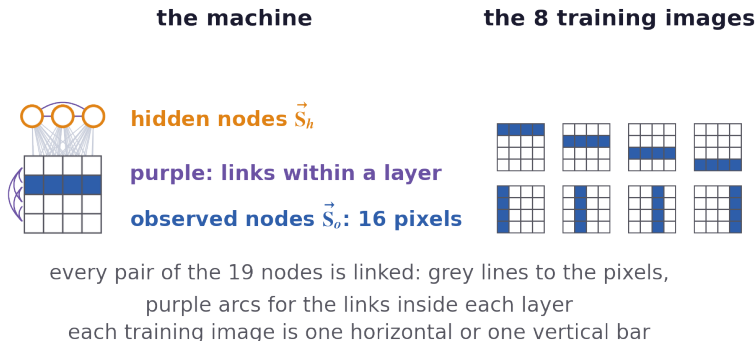


Figure 26: The running example for the Boltzmann machine: a machine with sixteen observed nodes, drawn as the pixels of a 4×4 image, and three hidden nodes above them. Every pair of the nineteen nodes is joined; the grey lines are the hidden-to-pixel weights and the purple arcs stand for the weights inside each layer. The training data are the eight images that contain exactly one horizontal or one vertical bar. A blue pixel, here and in the figures that follow, is one with $S = 1$.

The learning rule compares clamped and free correlations

The gradient of the KL divergence is remarkably simple, and it gives a one-line update. Each weight is changed by

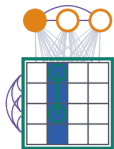
$$\omega_{ij} \mapsto \omega_{ij} + \Delta\omega_{ij},$$
$$\Delta\omega_{ij} = -\zeta \frac{\partial KL(\vec{\omega})}{\partial \omega_{ij}} = \frac{\zeta}{T} \left\{ \underbrace{\langle S_i S_j \rangle_{\text{clamped}}}_{\text{data clamped}} - \underbrace{\langle S_i S_j \rangle}_{\text{free running}} \right\},$$

where ζ is a small positive constant, the learning rate. The rule is gradient descent on the KL divergence.

The two averages are taken in two different regimes. In the clamped phase the observed nodes are held at a training example and only the hidden nodes are free to move, so the average is taken under $R(\vec{S}_o) P(\vec{S}_h | \vec{S}_o)$. In the free phase every node follows the model and the average is taken under $P(\vec{S})$. Each average is the correlation of the same pair of nodes i and j , measured under the two regimes.

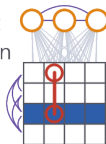
The learning rule compares clamped and free correlations

clamped phase



observed nodes held at
a training image; hidden
nodes sampled given it

free phase



every node sampled
from the model: the
machine dreams

filled or blue: a node with $S=1$

circled pair: $\langle S_i S_j \rangle_{\text{clamped}}$

circled pair: $\langle S_i S_j \rangle$

the same pair of pixels is watched in both phases

Figure 27: The two phases on the running example. In the clamped phase, the panel with the teal border, the sixteen pixels are held at one training image and the three hidden nodes are sampled given it. In the free phase, the panel beside it, every node is sampled from the model. A filled node or a blue pixel has $S = 1$. The circled pair is the same pair of pixels in both phases, and the rule compares how often that pair is on together in the two.

The first term is Hebbian and the second anti-Hebbian

Read the two terms as learning rules in their own right. The clamped term raises the weight between two nodes that are on together while the machine is looking at the data, which is Hebb's rule of the first section. The free term lowers the weight between two nodes that are on together while the machine is running by itself, which is the same rule with the opposite sign: anti-Hebbian.

The course notes offer a speculation about what this would mean for a brain. Hebbian learning would happen when the animal is awake and the observed nodes are driven by the senses; anti-Hebbian learning would happen when it is asleep and the machine is sampling from its own model, that is to say dreaming. It is a speculation, but it is a sharp one: the rule needs both phases and cannot work with either alone.

The first term is Hebbian and the second anti-Hebbian

The update vanishes when the two correlations are equal for every pair, which is a stationary point of the descent. For a pair of observed nodes that condition says the model has matched a pair statistic of the data; for a pair that involves a hidden node both sides are quantities of the model, since the data say nothing about the hidden nodes. Even a complete fit of the pair statistics is a complete fit of R only when the machine is rich enough to represent R .

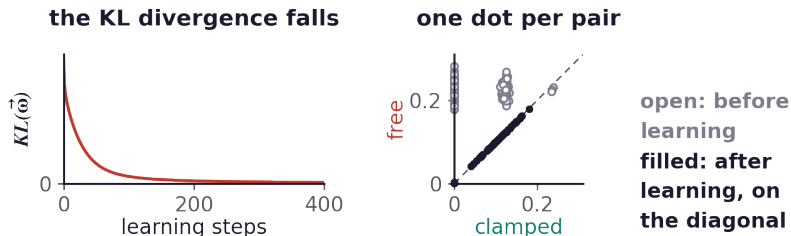
The first term is Hebbian and the second anti-Hebbian

The data distribution R is not usually known as a formula; what is available is a set of N training examples, so R is approximated by the distribution that puts weight $1/N$ on each of them,

$$R(\vec{S}_o) = \frac{1}{N} \sum_{n=1}^N \delta(\vec{S}_o - \vec{S}_o^n),$$

where $\vec{S}_o^n$ is the n -th training example and N is the number of them. This is the data-as-a-distribution of the first section, written for the observed nodes. Sampling from R is then nothing more than picking one of the training examples at random.

The first term is Hebbian and the second anti-Hebbian



each dot is one of the 171 pairs of nodes of the running example
learning moves every dot onto the diagonal: clamped equals free
both correlations are exact here: the free one over all 2^{19} states

Figure 28: Learning on the running example, with both correlations computed exactly: the free one by enumerating all 2^{19} states of the machine and the clamped one by enumerating the 2^3 hidden states of each training image. Here $\zeta = 0.5$, $T = 1$ and 400 steps. The first plot is the KL divergence, which falls from 8.77 to 0.086. In the second each dot is one of the 171 pairs of nodes, with its clamped correlation on the horizontal axis and its free correlation on the vertical one; the open dots are before learning and the filled ones after. The dots move sideways as well as upwards, because a pair that involves a hidden node has a clamped correlation that depends on the weights too.

Gibbs sampling estimates both correlations

The free correlation is a sum over all $2^{|\mathcal{V}|}$ states of the machine, so for a machine of any size it cannot be computed. The clamped one is cheaper, a sum over the $2^{|\mathcal{V}_h|}$ states of the hidden nodes for each training example, but it too becomes impossible once the hidden layer is large. Both are estimated by Gibbs sampling, using the average over M samples of a Gibbs chain,

$$\langle S_i S_j \rangle \approx \frac{1}{M} \sum_{s=1}^M S_i^s S_j^s,$$

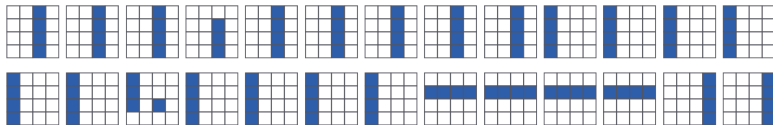
and the two phases differ only in how the samples are drawn. For the free correlation the chain runs on the whole machine: at each step one node is picked at random and redrawn from its own conditional $P(S_i | \vec{S}/i)$. For the clamped correlation a training example is drawn from R , the observed nodes are held at it, and only the hidden nodes are redrawn, each from its conditional given the rest, so the chain samples $P(\vec{S}_h | \vec{S}_o^n)$.

Gibbs sampling estimates both correlations

This is where the method becomes expensive. Each learning step needs two sets of samples, and each set needs a chain long enough to have forgotten where it started. The snapshots of the free chain show why that is slow here: the trained machine is a good model of the data, its distribution has eight sharp peaks, and the chain stays in one of them for a long time before a fluctuation large enough to move it to another. The course notes end the section with the verdict that Boltzmann machines are hard to use without extra ingredients.

Gibbs sampling estimates both correlations

samples of the observed nodes, every 250 steps



read left to right, top row first: 24 of the 26 snapshots are exact bar images
the chain moves from one bar to another only three times in 6500 updates

Figure 29: Twenty-six snapshots of the observed nodes of the trained machine, taken every 250 single-neuron updates of one free chain. Read left to right, top row first. Twenty-four of the snapshots are exact bar images, and over all 6500 updates, not only at the moments sampled here, the chain moves from one bar to another only three times.

The learning rule is the gradient of the KL divergence

The rule that compares the clamped and the free correlation is derived here, and then read as an optimisation. The slides compress this to three steps; the full derivation follows.

The learning rule is the gradient of the KL divergence

The quantity to minimise is $KL(\vec{\omega}) = \sum_{\vec{S}_o} R(\vec{S}_o) \log\{R(\vec{S}_o)/P(\vec{S}_o)\}$.

Only $P(\vec{S}_o)$ depends on the weights, and $\partial \log P / \partial \omega_{ij} = (1/P) \partial P / \partial \omega_{ij}$, so

$$\frac{\partial KL(\vec{\omega})}{\partial \omega_{ij}} = - \sum_{\vec{S}_o} \frac{R(\vec{S}_o)}{P(\vec{S}_o)} \frac{\partial P(\vec{S}_o)}{\partial \omega_{ij}}.$$

Everything now turns on the derivative of the marginal. Write it out,

$$P(\vec{S}_o) = \frac{1}{Z} \sum_{\vec{S}_h} \exp\{-E(\vec{S})/T\},$$

$$\frac{\partial P(\vec{S}_o)}{\partial \omega_{ij}} = \frac{1}{Z} \frac{\partial}{\partial \omega_{ij}} \sum_{\vec{S}_h} \exp\{-E(\vec{S})/T\} - P(\vec{S}_o) \frac{\partial \log Z}{\partial \omega_{ij}},$$

where the second term comes from differentiating the $1/Z$ in front.

The derivative of $\log Z$ is a correlation

Two derivatives are needed, and both come from the same small calculation.

In $E(\vec{S}) = -\frac{1}{2} \sum_{kl} \omega_{kl} S_k S_l$ the pair ij appears twice, as $\omega_{ij} S_i S_j$ and as $\omega_{ji} S_j S_i$, and the two weights are one parameter because the machine is symmetric. Differentiating with respect to that one parameter,

$$\frac{\partial}{\partial \omega_{ij}} \left(-\frac{E(\vec{S})}{T} \right) = \frac{S_i S_j}{T}.$$

The factor $\frac{1}{2}$ and the double counting cancel.

The derivative of $\log Z$ is a correlation

For the partition function, $Z = \sum_{\vec{S}} \exp\{-E(\vec{S})/T\}$, so

$$\frac{\partial \log Z}{\partial \omega_{ij}} = \frac{1}{Z} \sum_{\vec{S}} \frac{S_i S_j}{T} \exp\{-E(\vec{S})/T\} = \frac{1}{T} \sum_{\vec{S}} S_i S_j P(\vec{S}) = \frac{1}{T} \langle S_i S_j \rangle.$$

The derivative of the log partition function with respect to a weight is the correlation of the two neurons that weight joins, under the model. This is where the free term of the learning rule comes from, and it is the reason the rule needs a free phase at all.

The learning rule is the gradient of the KL divergence

Substituting both derivatives into the expression for $\partial P(\vec{S}_o)/\partial \omega_{ij}$,

$$\frac{\partial P(\vec{S}_o)}{\partial \omega_{ij}} = \frac{1}{T} \sum_{\vec{S}_h} S_i S_j P(\vec{S}) - P(\vec{S}_o) \frac{1}{T} \sum_{\vec{S}} S_i S_j P(\vec{S}),$$

where the first sum runs over the hidden states only, with $\vec{S}_o$ held at the value under consideration. Putting this back into the expression for $\partial KL(\vec{\omega})/\partial \omega_{ij}$, using $P(\vec{S})/P(\vec{S}_o) = P(\vec{S}_h | \vec{S}_o)$ in the first term and $\sum_{\vec{S}_o} R(\vec{S}_o) = 1$ in the second,

$$\frac{\partial KL(\vec{\omega})}{\partial \omega_{ij}} = -\frac{1}{T} \left\{ \underbrace{\sum_{\vec{S}} S_i S_j P(\vec{S}_h | \vec{S}_o) R(\vec{S}_o)}_{\text{from the sum over the hidden nodes}} - \underbrace{\sum_{\vec{S}} S_i S_j P(\vec{S})}_{\text{from log } Z} \right\}.$$

The learning rule is the gradient of the KL divergence

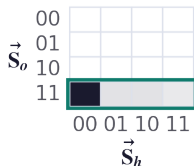
The first term is the correlation with the data clamped and the second the correlation of the free machine, so gradient descent, $\Delta\omega_{ij} = -\zeta \partial KL / \partial \omega_{ij}$, is exactly the rule that compares the clamped and the free correlation.

The learning rule is the gradient of the KL divergence

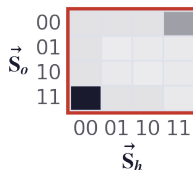
One sign is worth flagging. Equation (8) of the course notes carries a minus in front of the braces, which would make the clamped term lower the weight of a pair that is on together in the data. That contradicts the Hebbian reading that page 20 of those notes gives the same rule, and it comes from taking $\partial(-E/T)/\partial\omega_{ij} = -S_i S_j/T$ rather than $+S_i S_j/T$. These notes use the sign that this derivation gives, which agrees with Ackley, Hinton and Sejnowski (1985) and was checked here by finite differences on a machine of five nodes.

The learning rule is the gradient of the KL divergence

the clamped term



the free term



rows: the two observed nodes; columns: the two hidden nodes

darker means more probable; this machine has 2 observed, 2 hidden

the clamped term averages over the one row of $\vec{s}_o = 11$

the free term averages over all 16 states: that is where $\log Z$ comes from

Figure 30: Where the two terms come from, on a machine with two observed and two hidden nodes. Each table has the four states of the observed nodes down its rows and the four states of the hidden nodes across its columns, so one cell is one state of the whole machine and a darker cell is a more probable one. The clamped term averages over the single row of the training example, weighted by the conditional of the hidden nodes; the other rows are left blank because they take no part in that average, not because they are improbable. The free term averages over the whole table, which is the sum that $\log Z$ contributes.

Boltzmann machine learning is maximum likelihood

The rule was derived as gradient descent on a KL divergence. Splitting that divergence, exactly as in the first section, says what it really is:

$$KL(\vec{\omega}) = \underbrace{\sum_{\vec{S}_o} R(\vec{S}_o) \log R(\vec{S}_o)}_{\text{independent of } \vec{\omega}} - \sum_{\vec{S}_o} R(\vec{S}_o) \log P(\vec{S}_o).$$

The first term depends on the data alone, so only the second can be changed by learning. Substituting the training data for R ,

$$- \sum_{\vec{S}_o} R(\vec{S}_o) \log P(\vec{S}_o) = -\frac{1}{N} \sum_{n=1}^N \log P(\vec{S}_o^n),$$

which is minus the average log-likelihood of the training examples. Minimising the KL divergence is maximum likelihood.

Boltzmann machine learning is maximum likelihood

That is worth having for two reasons. It says the rule is the same criterion used everywhere else in this lecture. And maximum likelihood remains a sensible thing to do even when the model cannot represent R exactly: it then picks the member of the family that fits best.

Boltzmann machine learning is maximum likelihood

Saying that the criterion is maximum likelihood does not make it computable. Each term $\log P(\vec{S}_o^n)$ is the logarithm of a sum over the hidden states divided by Z , and Z is a sum over every state of the machine. For a machine of any useful size neither sum can be evaluated.

The shape of the difficulty is by now familiar: a log-likelihood with hidden variables summed out inside the logarithm. That is the problem EM was designed for, and the Boltzmann machine fits that framework exactly. Gibbs sampling then supplies the two expectations that the framework needs.

The Boltzmann machine learns by a generalised EM

Write the free energy of the EM section for this model. The hidden variables are the hidden nodes, one set per training example, so there is one distribution Q^n over $\vec{S}_h$ for each of the N training images, and the parameters are the weights:

$$\mathcal{F}(Q, \omega) = \underbrace{-\sum_{n=1}^N \log P(\vec{S}_o^n; \omega)}_{\text{minus the log-likelihood}} + \underbrace{\sum_{n=1}^N \sum_{\vec{S}_h^n} Q^n(\vec{S}_h^n) \log \frac{Q^n(\vec{S}_h^n)}{P(\vec{S}_h^n | \vec{S}_o^n; \omega)}}_{\text{KL divergences, never negative}}.$$

The semicolon before ω marks it as a parameter rather than a conditioning variable, which is the notation of the course notes; the bar is kept for conditioning.

Minimising over Q and ω is maximum likelihood

Minimising over the Q^n gives $Q^n(\vec{S}_h^n) = P(\vec{S}_h^n | \vec{S}_o^n; \omega)$, the clamped distribution of the hidden nodes, and then $\mathcal{F}$ is minus the log-likelihood. So minimising $\mathcal{F}$ over Q and ω together is maximum likelihood, and alternating the two minimisations is EM.

The M-step gradient is the learning rule with a clamped Q

The M-step minimises $\mathcal{F}$ over the weights with the Q^n held fixed. Dropping the terms that do not contain ω , it minimises

$$\sum_{n=1}^N \sum_{\vec{S}_h} Q^n(\vec{S}_h) \left\{ \frac{E(\vec{S}_o^n, \vec{S}_h)}{T} + \log Z(\omega) \right\},$$

because $-\log P(\vec{S}_o, \vec{S}_h; \omega) = E/T + \log Z$. Differentiating with respect to ω_{ij} , using $\partial E / \partial \omega_{ij} = -S_i S_j$ and the derivative $\partial \log Z / \partial \omega_{ij} = \langle S_i S_j \rangle / T$ of the partition function, and using $\sum_{\vec{S}_h} Q^n(\vec{S}_h) = 1$,

$$\frac{\partial \mathcal{F}}{\partial \omega_{ij}} = \frac{N}{T} \left\{ \langle S_i S_j \rangle_P - \langle S_i S_j \rangle_Q \right\},$$

where the first average is under the free machine and the second under the clamped Q , averaged over the training examples.

The M-step gradient is the learning rule with a clamped Q

A gradient step on this is the learning rule that compares the two correlations, up to the factor N : that rule averages over the training examples through R where this one sums over them, and the difference is absorbed into the learning rate.

The M-step gradient is the learning rule with a clamped Q

Page 27 of the course notes writes this M-step as an arg min of $\sum Q^n \log P$, which needs an arg max; its energy term also lacks the sign, the factor $1/T$ and the factor N in front of $\log Z$.

The Boltzmann machine learns by a generalised EM

The pieces now fit together. The E-step computes, for each training image, the distribution of the hidden nodes with the observed nodes clamped to that image. The M-step does not minimise $\mathcal{F}$ over the weights exactly, which would be a problem as hard as the original one; it takes a few steps of the learning rule, which lower $\mathcal{F}$ without minimising it, provided the learning rate is small enough that a step does not overshoot. That is generalised EM, and the inequality of the EM section still holds.

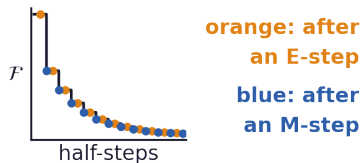
Two warnings come with it. Every step of either kind needs Gibbs sampling, so the quantities used are estimates; with sampled correlations $\mathcal{F}$ falls only on average, not at every step. And EM converges to a local optimum, which need not be the maximum likelihood estimate. The course notes also replace each Q^n by a product of factors, one per hidden node, which is the mean field approximation of the last lecture; the E-step is then no longer exact and $\mathcal{F}$ becomes an upper bound.

The Boltzmann machine learns by a generalised EM

EM for the machine



$\mathcal{F}$ falls



12 iterations on the running example, each M-step ten gradient steps
the E-steps change $\mathcal{F}$ little; the M-steps do most of the work
exact correlations and a small step: $\mathcal{F}$ falls at every half-step

Figure 31: EM on the running example: twelve iterations, each M-step ten gradient steps with $\zeta = 0.5$. The staircase is the free energy after every half-step, an orange dot after each E-step and a blue dot after each M-step; it falls from 86.8 to 19.8. The correlations here are computed exactly and the learning rate is small enough not to overshoot, so every half-step lowers $\mathcal{F}$; the E-steps change it little and the M-steps do most of the work.

The restricted Boltzmann machine has a bipartite graph

The extra ingredient that makes these machines usable is a restriction on the graph. A restricted Boltzmann machine (RBM) keeps the weights between the observed and the hidden nodes and removes the weights inside each layer. The graph is then bipartite: every weight joins a node of one layer to a node of the other, and none joins two nodes of the same layer. Write $\mathcal{V}_o$ for the set of observed nodes and $\mathcal{V}_h$ for the set of hidden ones; the energy is then

$$E(\vec{S}) = - \sum_{i \in \mathcal{V}_o, j \in \mathcal{V}_h} \omega_{ij} S_i S_j,$$

with no factor $\frac{1}{2}$ because this sum visits each pair once.

The point of the restriction is that the nodes within each layer become independent of one another once the other layer is held fixed,

$$P(\vec{S}_o | \vec{S}_h) = \prod_{i \in \mathcal{V}_o} P(S_i | \vec{S}_h), \quad P(\vec{S}_h | \vec{S}_o) = \prod_{j \in \mathcal{V}_h} P(S_j | \vec{S}_o).$$

Why the RBM conditional factorises

The factorisation is a consequence of the missing weights. With $\vec{S}_h$ held fixed, the energy is

$$E(\vec{S}) = - \sum_{i \in \mathcal{V}_o} S_i \left(\sum_{j \in \mathcal{V}_h} \omega_{ij} S_j \right),$$

a sum of terms each of which contains a single observed node. No term couples two observed nodes, because those weights were removed. The exponential of a sum is a product, so $\exp\{-E/T\}$ factorises over the observed nodes, and normalising a product of independent factors gives a product of normalised factors.

Why the RBM conditional factorises

Each factor is the sigmoid conditional of a single neuron, with the sum running over the hidden layer only,

$$P(S_i | \vec{S}_h) = \frac{1}{Z_i} \exp \left\{ \frac{1}{T} S_i \sum_{j \in \mathcal{V}_h} \omega_{ij} S_j \right\},$$

$$Z_i = \sum_{S_i \in \{0,1\}} \exp \left\{ \frac{1}{T} S_i \sum_{j \in \mathcal{V}_h} \omega_{ij} S_j \right\}.$$

The normalisation Z_i is a sum over the two values of one neuron, which is why it is computable while Z is not. The same argument with the layers exchanged gives $P(\vec{S}_h | \vec{S}_o)$.

The restricted Boltzmann machine has a bipartite graph

Boltzmann machine



purple: weights within a layer weights only between the layers

blue nodes: the observed layer; white nodes: the hidden layer
dropping the within-layer weights is what makes each conditional factorise

restricted machine



Figure 32: A general Boltzmann machine and a restricted one. The blue nodes are the observed layer and the white ones the hidden layer; the grey lines are the weights between the layers and the purple arcs the weights inside a layer, which the restricted machine does not have.

RBM sample a whole layer at once, and can be stacked

The factorisation changes the cost of sampling. In the clamped phase the observed nodes are fixed at a training image, and the hidden layer can be drawn in a single step, because its nodes are independent given that image: no chain is needed at all. In the free phase the chain alternates, drawing the hidden layer from the observed one and then the observed layer from the hidden one, and each of those steps is one pass over a layer rather than one neuron at a time.

Each step is therefore cheap. The chain may still need many alternations to move between distant states, so it still mixes slowly, which is to say it still takes many steps to cross from one peak of the distribution to another. The usual further ingredient is contrastive divergence, which starts the free chain at a training image rather than at a random state and runs the alternation for only a few steps.

RBM sample a whole layer at once, and can be stacked

One RBM learns one layer of features from the data. Hinton, Osindero and Teh (2006) stacked them: train an RBM on the images, then treat the activities of its hidden layer as the data for a second RBM, and so on. Each machine in the stack learns features of the features below it, and the result is a deep network, trained one layer at a time without labels.

On images of handwritten digits the stacked network learnt a generative model of the images and their labels together, with the labels attached only at the top layer and the layers below trained without them, and classified the digits better than the best back-propagation networks and support vector machines of the time.

RBM's sample a whole layer at once, and can be stacked

sample a whole layer at once

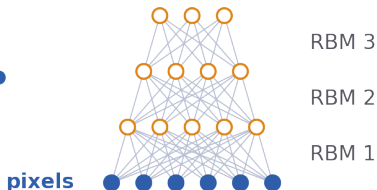


orange: $P(\vec{S}_h | \vec{S}_o)$

blue: $P(\vec{S}_o | \vec{S}_h)$

time runs to the right

stacked, layer by layer



each alternation resamples one whole layer, so a step is cheap
the free chain may still need many alternations to move far
in the stack each machine is trained on the hidden layer below it

Figure 33: Alternating sampling and stacking. In the sampling panel time runs to the right and each pair of arrows is one alternation: orange draws the hidden layer from the observed one, blue draws the observed layer from the hidden one. In the stacking panel the bottom row is the pixels and each pair of adjacent rows is one RBM of the stack, trained on the layer below it.

Summary

- ▶ Learning estimates the parameters of a distribution from data, by maximum likelihood, in batch or online.
- ▶ Maximum likelihood minimises the KL divergence to the data; a prior matters most when there are few datapoints.
- ▶ Hidden variables are the ones the training data leave unspecified; k-means alternates assignments and means.
- ▶ EM learns with hidden variables: the E-step infers them, the M-step re-estimates the parameters.
- ▶ EM minimises a free energy; with exact E-steps every iteration leaves the log-likelihood at least as high, up to a local optimum.

Summary

- ▶ Soft k-means is EM for a mixture of Gaussians, and hard k-means is its zero-variance limit.
- ▶ The Boltzmann machine is a Gibbs distribution on binary neurons; Gibbs sampling estimates its correlations.
- ▶ Its rule, clamped minus free correlations, is maximum likelihood carried out by a generalised EM.

References

- [1] A. L. Yuille. *Probabilistic Models of the Visual Cortex*: lecture notes, Johns Hopkins University, 2025.
- [2] D. O. Hebb. *The Organization of Behavior*. Wiley, 1949.
- [3] E. R. Kandel. The molecular biology of memory storage: a dialogue between genes and synapses. *Science*, 294(5544):1030–1038, 2001.
- [4] H. Robbins and S. Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [5] T. L. Griffiths, N. Chater and J. B. Tenenbaum. *Bayesian Models of Cognition: Reverse Engineering the Mind*. MIT Press, 2024.
- [6] A. P. Dempster, N. M. Laird and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society B*, 39(1):1–38, 1977.
- [7] R. M. Neal and G. E. Hinton. A view of the EM algorithm that justifies incremental, sparse, and other variants. In *Learning in Graphical Models*, Kluwer, 1998, pp. 355–368.

References

- [8] D. Arthur and S. Vassilvitskii. k-means++: the advantages of careful seeding. *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035, 2007.
- [9] Q. Yu, H. Wang, S. Qiao, M. Collins, Y. Zhu, H. Adam, A. Yuille and L.-C. Chen. k-means mask transformer. *European Conference on Computer Vision*, 2022.
- [10] J. Hertz, A. Krogh and R. G. Palmer. *Introduction to the Theory of Neural Computation*. Addison-Wesley, 1991.
- [11] A. Banerjee, I. S. Dhillon, J. Ghosh and S. Sra. Clustering on the unit hypersphere using von Mises-Fisher distributions. *Journal of Machine Learning Research*, 6:1345–1382, 2005.
- [12] D. H. Ackley, G. E. Hinton and T. J. Sejnowski. A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1):147–169, 1985.
- [13] Y. Zhang, X. Li, J. M. Samonds and T. S. Lee. Relating functional connectivity in V1 neural circuits and 3D natural scenes using Boltzmann machines. *Vision Research*, 120:121–131, 2016.
- [14] G. E. Hinton, S. Osindero and Y.-W. Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554, 2006.